



Mini-informe Stop & Wait

Autor: Sebastian Cadena

Profesor: Ivana Bachmann

Repositorio: https://github.com/JuniorC-code/redes/tree/main/actividad_3-STOP%26WAIT

El protocolo de control de errores *Stop & Wait* es utilizado para asegurarse de que los paquetes enviados por el emisor lleguen al receptor, realizando reenvíos de paquetes en caso de que alguno se pierda. En *Stop & Wait*, cuando el emisor envía un paquete, este se queda esperando a que el receptor le confirme que dicho paquete llegó mediante un mensaje ACK. Una vez el emisor recibe el mensaje ACK del receptor, este procede a enviar el siguiente paquete de información. El emisor esperará el mensaje ACK por un periodo de tiempo que llamaremos '*timeout*', en caso de no recibir el mensaje ACK que esperaba dentro del periodo de timeout, el emisor reenviará el paquete al receptor.

Para nuestra actividad se programó una clase **socketTCP** la cual implementa sockets orientados a conexión usando **Stop & Wait**. Esta clase funciona de forma similar a los sockets orientados a conexión de python. **Además,** se implementó un cliente y un servidor que usan estos tipos de objetos **socketTCP** para poder comunicarse.

Las dificultades de la actividad residían en la implementación de **TCP simplificado con Stop & Wait,** también usando timeouts, mensajes ACK's entre otros...

La prueba final se basa en que nuestro cliente sea capaz de enviar un archivo a una dirección (IP, Puerto) donde el servidor escuchara. Por ejemplo, si queremos enviar desde el cliente el archivo **'archivo.txt'** a la dirección '**(localhost, 8000)**' veríamos algo como:

```
% python3 cliente.py localhost 8000 < archivo.txt
```

Código de client:

```python
import sys
import socketTCP

buffer = 21
address = ("localhost", 8000)


#APARTADO DE PRUEBAS ANTERIORES

# CLIENT

#client_socketTCP = socketTCP.SocketTCP()
#client_socketTCP.connect(address)
## test 1
#message = "Mensje de len=16".encode()
#client_socketTCP.send(message)
## test 2
#message = "Mensaje de largo 19".encode()
#client_socketTCP.send(message)
## test 3
#message = "Mensaje de largo 19".encode()
#client_socketTCP.send(message)
#client_socketTCP.recv_close()

##############################


if __name__ == "__main__":

    # Obtener argumentos:
    # python3 cliente.py localhost 8000 < archivo.txt
    host = sys.argv[1]
    port = int(sys.argv[2])

    address = (host, port)

    # Crear SocketTCP
    client_socketTCP = socketTCP.SocketTCP()

    # Conectarse al servidor
    client_socketTCP.connect(address)

    # Leer todo el archivo desde stdin
    message = sys.stdin.buffer.read()

    # Enviar el contenido utilizando nuestro protocolo
    client_socketTCP.send(message)

    # Cerrar conexión

    client_socketTCP.close()
```

Código server:

```python
import socket
import socketTCP

server_address = ("localhost", 8000)


# APARTADO DE PRUEBAS

# SERVER

#server_socketTCP = socketTCP.SocketTCP()
#server_socketTCP.bind(server_address)
#connection_socketTCP, new_address = server_socketTCP.accept()#
## test 1
#buff_size = 16
#full_message = connection_socketTCP.recv(buff_size)
#print("Test 1 received:", full_message)
#if full_message == "Mensje de len=16".encode(): print("Test 1: Passed")
#else: print("Test 1: Failed")#
## test 2
#buff_size = 19
#full_message = connection_socketTCP.recv(buff_size)
#print("Test 2 received:", full_message)
#if full_message == "Mensaje de largo 19".encode(): print("Test 2: Passed")
#else: print("Test 2: Failed")#
## test 3
#buff_size = 14
#message_part_1 = connection_socketTCP.recv(buff_size)
#message_part_2 = connection_socketTCP.recv(buff_size)
#print("Test 3 received:", message_part_1 + message_part_2)
#if (message_part_1 + message_part_2) == "Mensaje de largo 19".encode(): print("Test 3: Passed")
#else: print("Test 3: Failed")#
#connection_socketTCP.close()

####################

if __name__ == "__main__":

    # Crear socket TCP
    server_socketTCP = socketTCP.SocketTCP()

    # Bind al puerto 8000
    server_socketTCP.bind(server_address)

    # Esperar conexión
    connection_socketTCP, new_address = server_socketTCP.accept()

    # Recibir el archivo completo
    message = b""

    while True:

        part = connection_socketTCP.recv(1024)

        if not part:
            break

        message += part

        # Si se recibió menos que el buffer solicitado,
        # probablemente ya recibimos todo.

    print(message.decode())

    # Cerrar conexión
    connection_socketTCP.recv_close()
```

Código de clase SocketTCP y decisiones de diseño:

```python
class SocketTCP():
    def __init__(self):
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        self.dest_adr = None
        self.seq_num = 0
        self.timeout = 1.0  # Timeout para retransmisión
        self.buffer_size_UDP = 21  # Tamaño del buffer para recibir mensajes

        # parte 5
        self.remaining_bytes = 0
        self.pending_data = b""

        # parte 7 --> Añadir memoria de la direccion donde llegaban los handshake, cosa de poder
        # mirar cuando falle el caso borde ultimo seg ACK no llega.

        self.handshake_addr = None


        self.fin_segment_received = None
```

Inicialmente, se intento añadir la menor cantidad de atributos a la clase por simplicidad, sin embargo para las pruebas finales y casos donde había leve asincronía entre el servidor y cliente, se añadió el atributo fin_segment_received para marcar la recepción de segmentos con las flags de FIN activadas.

Para la creación de los segmentos a enviar, se decidió por utilizar un byte donde se prendían bits marcando flags de dicho paquete mandado:

```python
def create_segment(self, ack: bool, syn: bool, fin: bool, seq: int, payload: bytes):

    # Enseguida crearemos los bits segun flags y lo añadiremos a el byte que representa a estos.

    flags = 0b00000000

    if ack:
        flags |= 0b00000001

    if syn:
        flags |= 0b00000010

    if fin:
        flags |= 0b00000100

    header = flags.to_bytes(1, byteorder="big") + seq.to_bytes(4, byteorder="big")

    # Devolvemos el segmento TCP como la concatenación del header y el payload

    return header + payload

def parse_segment(self, segment: bytes):
    # Extraemos el header y el payload del segmento
    header = segment[:5]
    payload = segment[5:]

    # Obtenemos los flags y el número de secuencia del header
    flags = header[0]
    seq = int.from_bytes(header[1:5], byteorder="big")

    # Determinamos el estado de cada flag
    ack = bool(flags & 0b00000001)
    syn = bool(flags & 0b00000010)
    fin = bool(flags & 0b00000100)

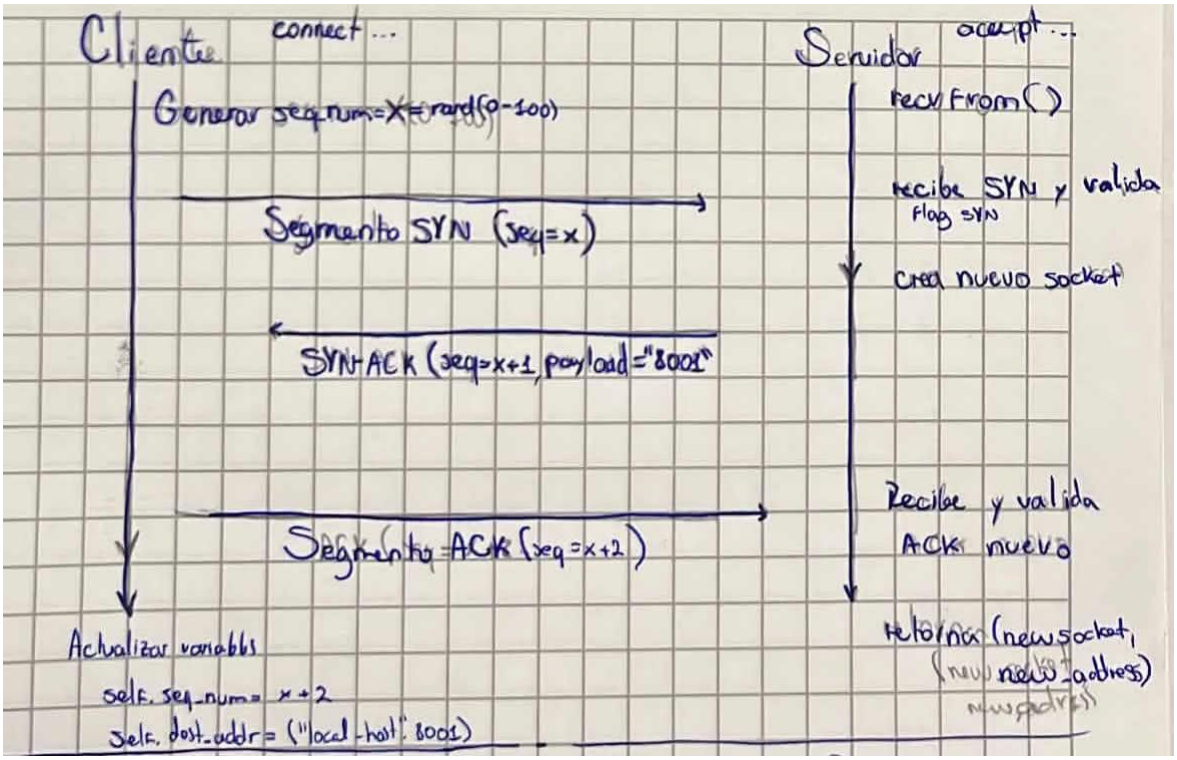
    # Devolvemos un diccionario con la información del segmento
    return {
        "ACK": ack,
        "SYN": syn,
        "FIN": fin,
        "seq": seq,
        "payload": payload
    }
```

De esta forma, se usan 5 bytes; 1 para guardar todos los flags necesarios para definir el tipo de paquete, y los otros 4 para el numero de secuencia del paquete a mandar o recibir.

Implementación de 3-way Handshake

Implementación del *3-way handshake* para que el emisor le avise al receptor que va a comenzar el envío de mensajes.

El código es relativamente fácil de entender, la forma de implementarlo se basa en el siguiente diagrama, que logra conectar al cliente con el servidor mediante una seguidilla de paquetes para coordinar el handshake:



Cierre de conexiones:

```python
# Parte 8: Fin de conexion para liberar recursos...

# La función close() debe implementar el cierre de conexión desde el lado del "Host A" según lo visto en el video.
# HOST A --> manda fin seq z, recibe fin+ack z+1 y devuelve ack z+2
def close(self):
    # primero creemos el mensaje FIN,  El FIN usa el seq_num actual como z
    FIN_segment = self.create_segment(ack=False, syn=False, fin=True, seq=self.seq_num, payload=b"")


    # parte 8: esperar 3 veces recepcionar mensaje FIN+ACK y si no cerrar conexion c:
    for intento in range(3):
        self.sock.sendto(FIN_segment, self.dest_adr)

        # esperar FIN+ACK en timeout
        self.sock.settimeout(self.timeout)

        try:
            mensaje, address = self.sock.recvfrom(self.buffer_size_UDP)
            mensaje_parseado = self.parse_segment(mensaje)


            if (mensaje_parseado["FIN"] and mensaje_parseado["ACK"] and mensaje_parseado["seq"] == self.seq_num + 1):
                # Recibimos correctamente FIN+ACK
                ACK_segment = self.create_segment(ack=True,syn=False,fin=False,seq=self.seq_num + 2,payload=b"")

                # Mandar el ACK 3 veces
                for _ in range(3):
                    self.sock.sendto(ACK_segment, self.dest_adr)

                    # Esperar timeout entre cada envío
                    self.sock.settimeout(self.timeout)

                    try:
                        self.sock.recvfrom(self.buffer_size_UDP)
                    except (socket.timeout,ConnectionResetError):
                        pass

                self.sock.close()
                print("conexion cerrada")
                return

        except socket.timeout:
            print("Timeout: reenviando FIN")

       # Llegamos al tercer timeout
    print("No se recibio FIN+ACK despues de 3 timeouts")
    print("Asumiendo que la contraparte se cerro")

    self.sock.close()
```

```python
def recv_close(self):

# Recibir FIN seq=z

    mensaje, address = self.sock.recvfrom(self.buffer_size_UDP)
    mensaje_parseado = self.parse_segment(mensaje)

    if mensaje_parseado["FIN"]:

        # Guardamos el seq del FIN recibido
        seq_fin = mensaje_parseado["seq"]

        # HOST B: responder FIN+ACK seq=z+1

        FIN_ACK_segment = self.create_segment(ack=True,syn=False,fin=True,seq=seq_fin + 1,payload=b"")

        self.sock.sendto(FIN_ACK_segment, self.dest_adr)

        # Esperar ACK seq=z+2 hasta 3 timeouts
        for intento in range(3):

            self.sock.settimeout(10)

            try:
                mensaje, address = self.sock.recvfrom(self.buffer_size_UDP)
                mensaje_parseado = self.parse_segment(mensaje)

                if (mensaje_parseado["ACK"] and mensaje_parseado["seq"] == seq_fin + 2):
                    # Cierre completado
                    self.sock.close()
                    print("conexion cerrada")
                    return

            except socket.timeout:
                print(f"Timeout {intento + 1}/3 esperando ACK final")

    # Después del tercer timeout asumimos que la contraparte se cerró
    print("No se recibio ACK final. Asumiendo que la contraparte se cerro.")

    self.sock.close()
```

Se añadieron las pruebas necesarias (provistas en el código de cliente y servidor también) para el manejo de perdida de paquetes, reenviándolos según corresponda si es que no recibe una confirmación. Es notable todo esto en el siguiente diagrama:

